



Network Swiss Army Knife (NSAK)

Bachelor Thesis – Integrating Agentic AI for Network Reconnaissance

June 18, 2026

Lukas von Allmen and Frank Gauss | Bern University of Applied Sciences

Overview

- ▶ Introduction
- ▶ Implementation
- ▶ Evaluation Setup
- ▶ AI Integration
- ▶ Results
- ▶ Discussion and Conclusion
- ▶ Demo: Agentic Reconnaissance in Action

Introduction

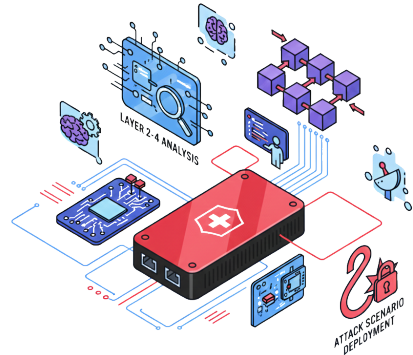
Starting Point: The NSAK Framework

Established in the predecessor project (Project 2)

A modular, open-source network security framework

Built around four reusable resource types:

-  **Device** *execution host at a network position*
-  **Environment** *target network topology*
-  **Scenario** *a red/blue team use case*
-  **Drill** *reusable building block*



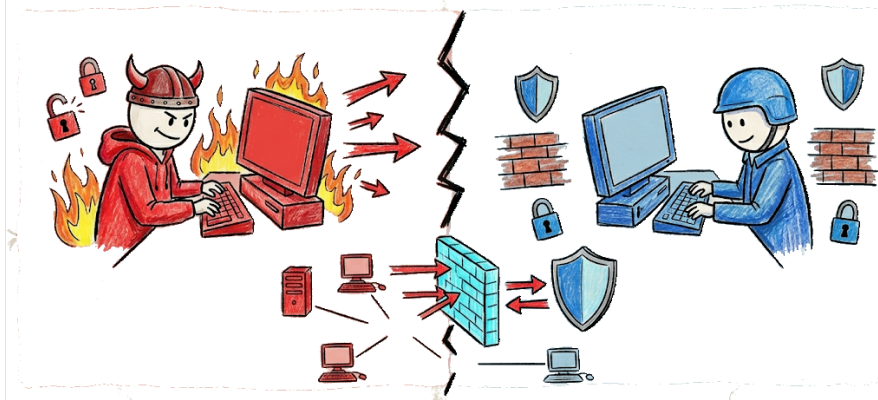
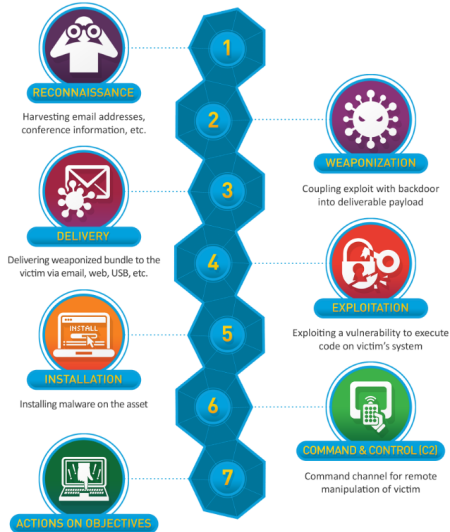


Figure: Red and Blue Team

Red Team



Blue Team

- Gather baseline data
- Detect
- Alert
- Investigate
- Plan a response
- Execute

Reconnaissance

The first stage of the cyber kill chain



Gathering information about a target network *before* any attack – it builds the map every later stage relies on, and is the focus of this thesis.

-  **Host discovery** *which systems are reachable on the network*
-  **Service & port enumeration** *what is running and exposed*
-  **Characterization** *OS, versions and vulnerability indicators*

Hypothesis and Research Questions

From static scenarios to agentic AI

Hypothesis

Integrating **agentic AI** into NSAK improves **adaptability, flexibility** and **automation** over static scenarios, while **lowering the operator's required expertise** to interpret results and plan follow-up steps.

Hypothesis and Research Questions

From static scenarios to agentic AI

Hypothesis

Integrating **agentic AI** into NSAK improves **adaptability, flexibility** and **automation** over static scenarios, while **lowering the operator's required expertise** to interpret results and plan follow-up steps.

- RQ1** How does an agentic AI scenario perform **compared to a static reference** scenario in network reconnaissance?
- RQ2** To what extent can agentic AI **support operators** during interactive reconnaissance?
- RQ3** To what extent is the implementation **suitable for real-world** deployment?

Project Goals

Order of implementation: Must → Should → Could

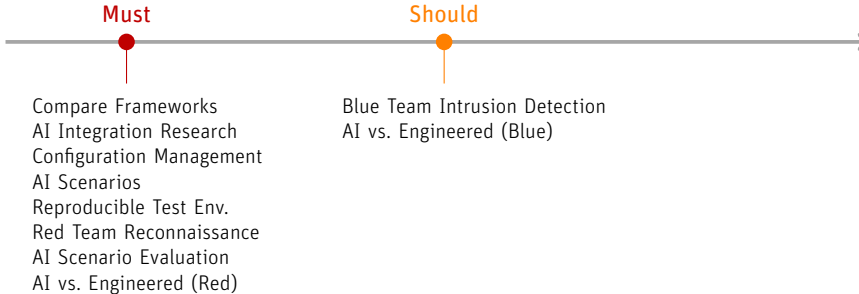
Must



Compare Frameworks
AI Integration Research
Configuration Management
AI Scenarios
Reproducible Test Env.
Red Team Reconnaissance
AI Scenario Evaluation
AI vs. Engineered (Red)

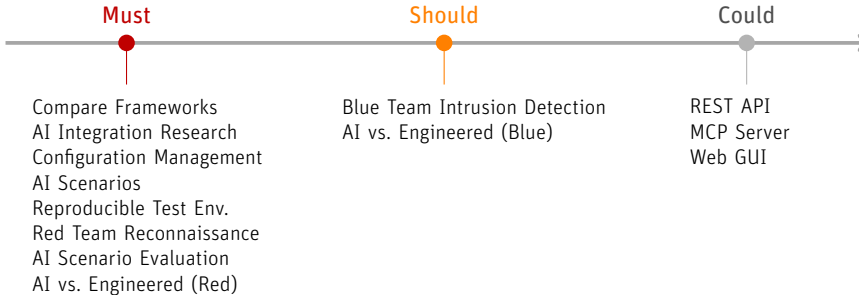
Project Goals

Order of implementation: Must → Should → Could



Project Goals

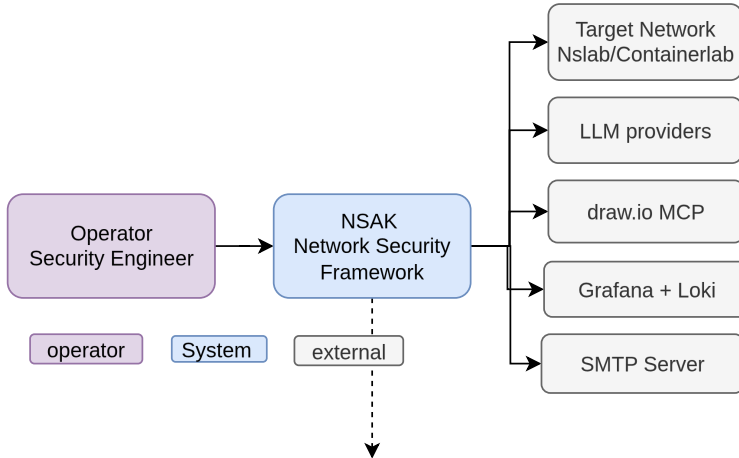
Order of implementation: Must → Should → Could



Implementation

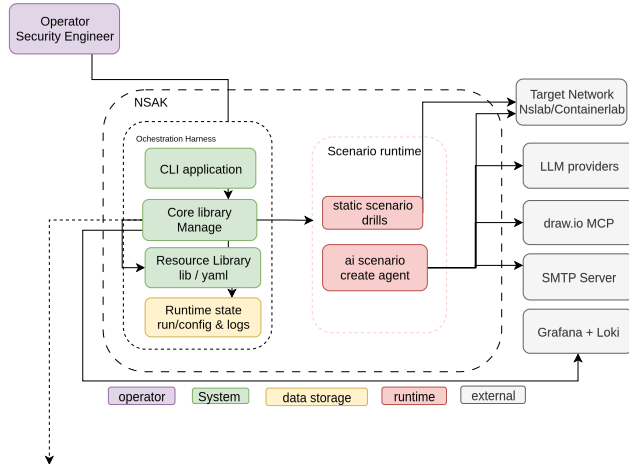
Layered Architecture

system-diagram



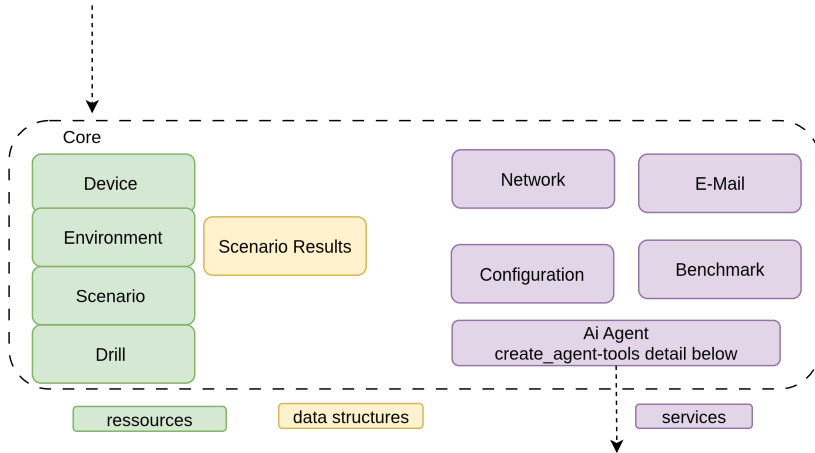
Layered Architecture

Lean Component Diagram



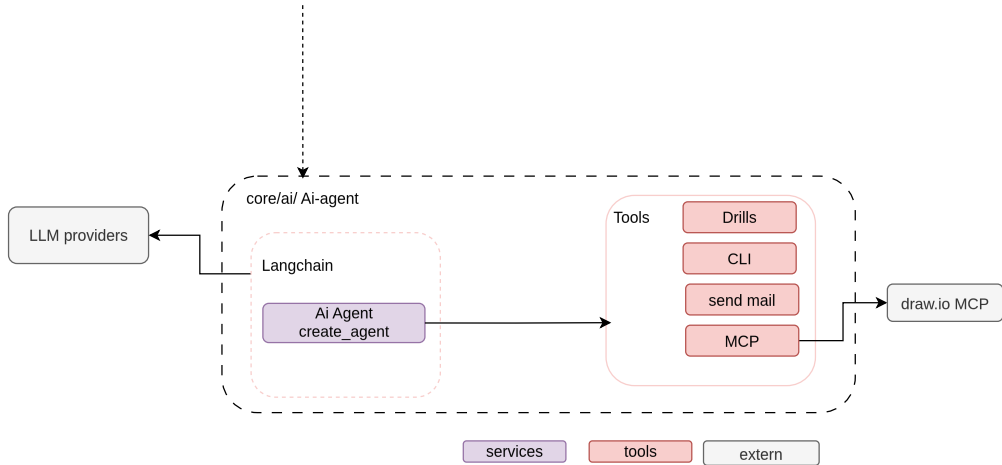
Layered Architecture

Core Diagram and Agent



Layered Architecture

Core Diagram and Agent



Evaluation Setup

Methology

3 LLMs • 2 Network Environments • Static Reference Baseline

LLMs

- qwen3: Self-hosted via Ollama
- gpt-oss: BFH-TI MLMP
- claude-opus-4-7: Anthropic API

Methology

3 LLMs • 2 Network Environments • Static Reference Baseline

LLMs

- qwen3: Self-hosted via Ollama
- gpt-oss: BFH-TI MLMP
- claude-opus-4-7: Anthropic API

Network Environments

- Reproducible Containerlab Network
- BFH Network Security LAB

Methology

3 LLMs • 2 Network Environments • Static Reference Baseline

LLMs

- qwen3: Self-hosted via Ollama
- gpt-oss: BFH-TI MLMP
- claude-opus-4-7: Anthropic API

Network Environments

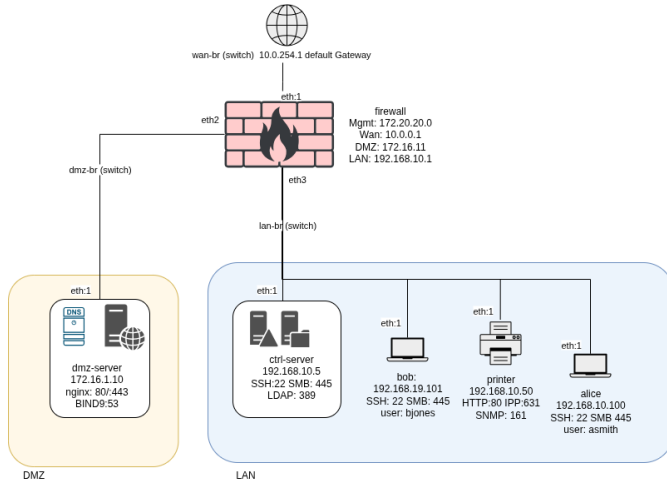
- Reproducible Containerlab Network
- BFH Network Security LAB

Reconnaissance Scenario

- Accepts an interface and/or a subnet as arguments
- discover_network_interfaces:
Iterates all interfaces if none is provided
- dhcp_request: Gets an IP via DHCP if needed
- discover_hosts: arp-scan & nmap
- enumerate_services: nmap NSE scripts
- Returns a ReconnaissanceScenarioResult

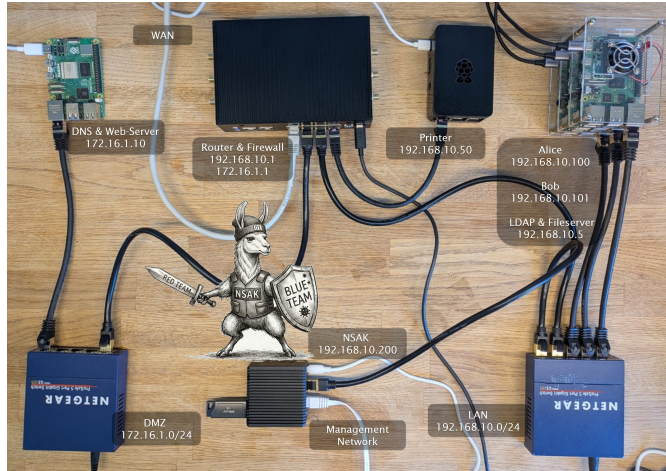
Reproducible Test Environment

A virtual enterprise network built with Containerlab



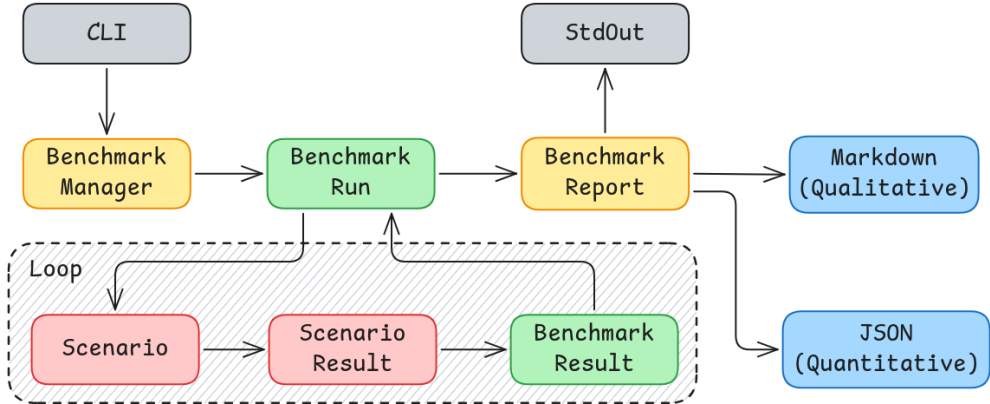
Physical Lab

The same network implemented with real hardware



Benchmarking Suite

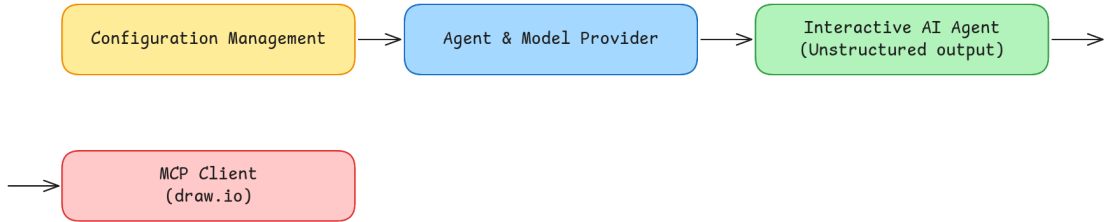
Automated test runs for non-interactive scenarios



AI Integration

AI Integration

Tool Calling: MCP Client



Problem 1: MCP Overhead

The problem

- MCP servers like draw.io provide a lot of tools
- Each tool, and its description contributes to the context
- Setups with small context windows are getting overwhelmed and do not call tools at all

Problem 1: MCP Overhead

The problem

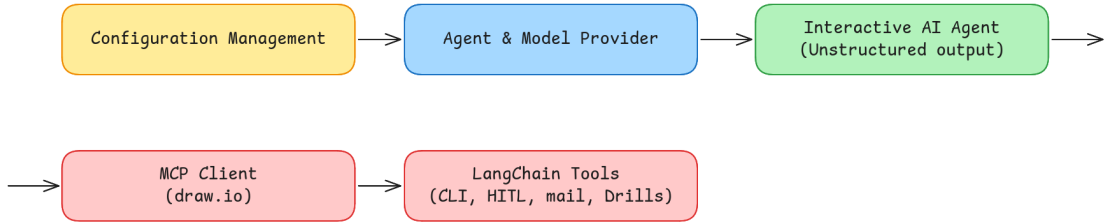
- MCP servers like draw.io provide a lot of tools
- Each tool, and its description contributes to the context
- Setups with small context windows are getting overwhelmed and do not call tools at all

Our solution

- Disable the MCP based tools per default
- Implement tools as simple python functions with LangChain's @tool decorator

AI Integration

Tool Calling: LangChain Tools



Configuration

Abstractions

Tool Calling

Scenarios

Problem 2: Too Many Tools

The problem

- There are a lot of drills which where automatically registered as tools
- Each tool, and it's description contributes to the context
- Setups with small context windows are getting overwhelmed and do not call tools at all

Problem 2: Too Many Tools

The problem

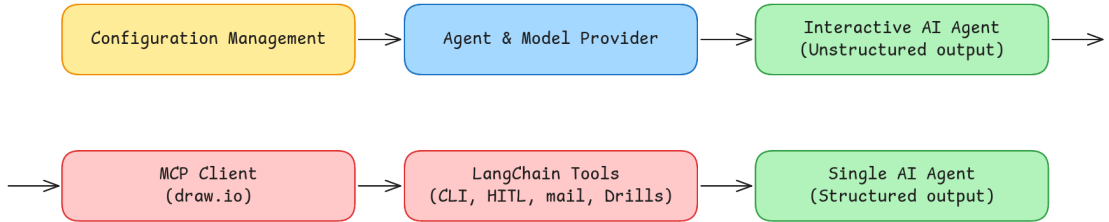
- There are a lot of drills which where automatically registered as tools
- Each tool, and it's description contributes to the context
- Setups with small context windows are getting overwhelmed and do not call tools at all

Our solution

- Use only a small set of tools
- Investigate “Dynamic Tool Selection” (not finished)

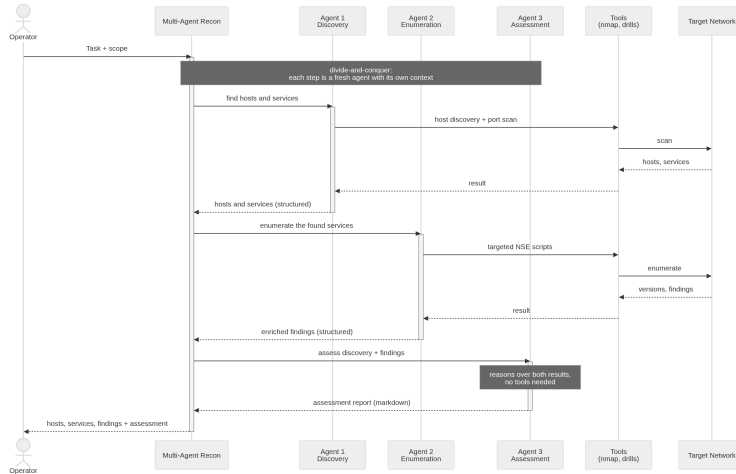
AI Integration

Scenario: Single AI Agent (Structured output)



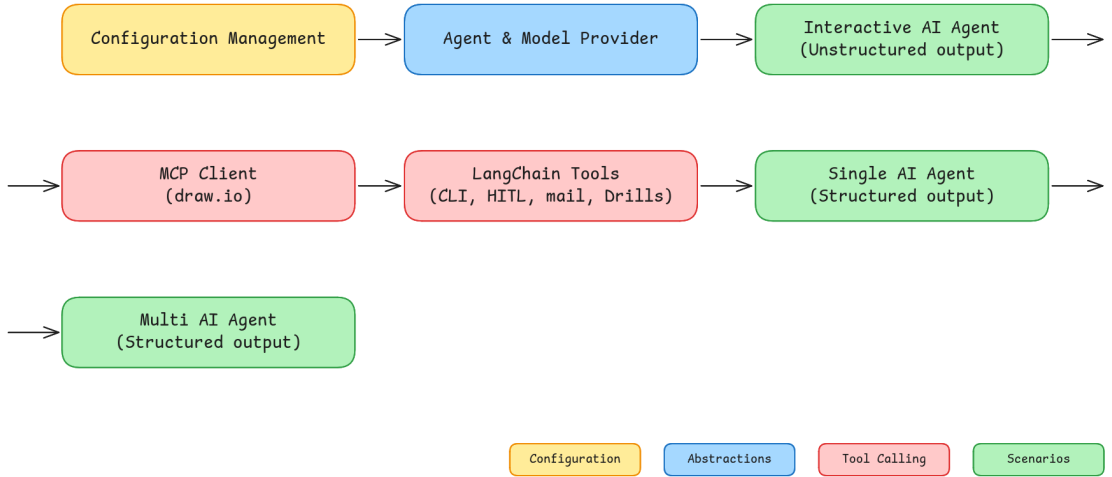
Problem 3: Taming the Context

Discovery → enumeration → assessment, each a fresh agent



AI Integration

Scenario: Multi AI Agent (Structured output)



Problem 4: Structured Output Complexity

The problem

- The failure rate skyrocketed in the BFH network security lab
- Not all models/providers support structured output equally well
- Native: Data structures are converted into grammars used in token sampling
- Tool Calling: Data structures are converted into tools

Problem 4: Structured Output Complexity

The problem

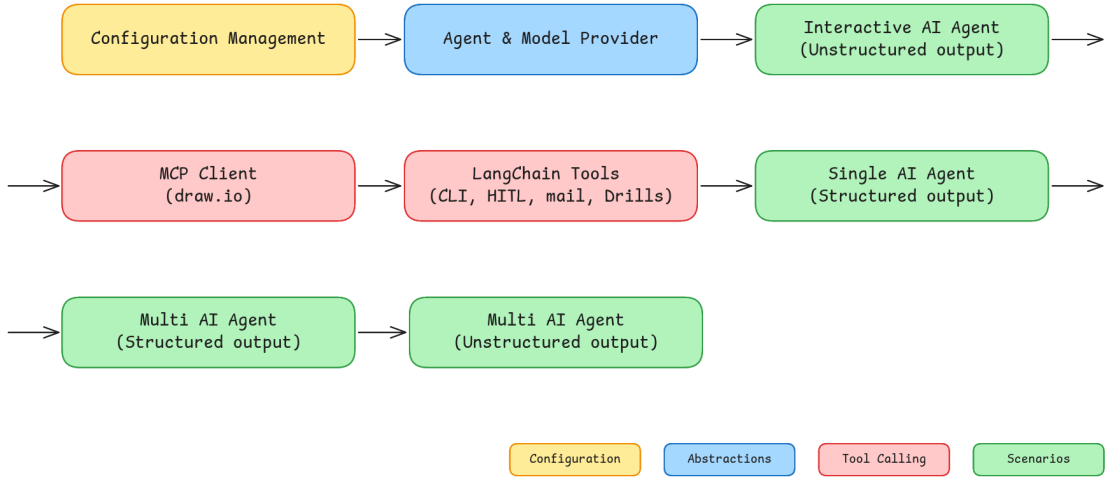
- The failure rate skyrocketed in the BFH network security lab
- Not all models/providers support structured output equally well
- Native: Data structures are converted into grammars used in token sampling
- Tool Calling: Data structures are converted into tools

Our solution

- Simplifying the data structure also simplifies the grammar/tool (was not enough)
- We implemented an AI scenario without structured output on the spot

AI Integration

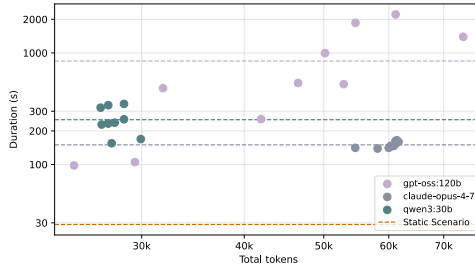
Scenario: Multi AI Agent (Unstructured output)



Results

Containerlab – Multi-Agent vs Single-Agent

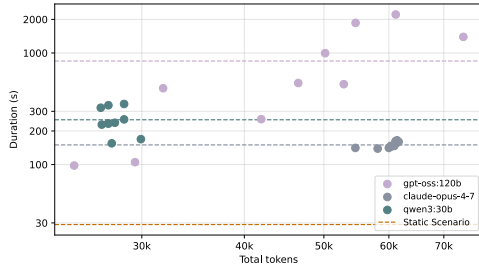
Tokens vs. duration – multi-agent first, then single-agent



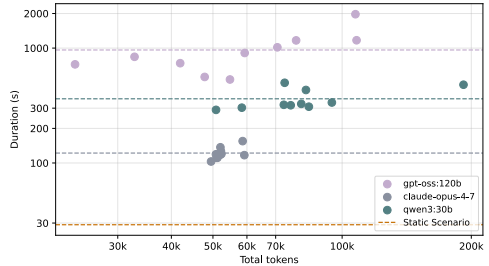
Multi-agent

Containerlab – Multi-Agent vs Single-Agent

Tokens vs. duration – multi-agent first, then single-agent



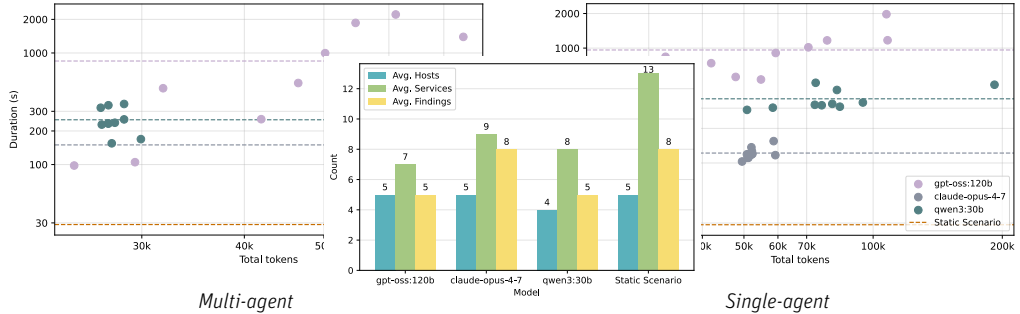
Multi-agent



Single-agent

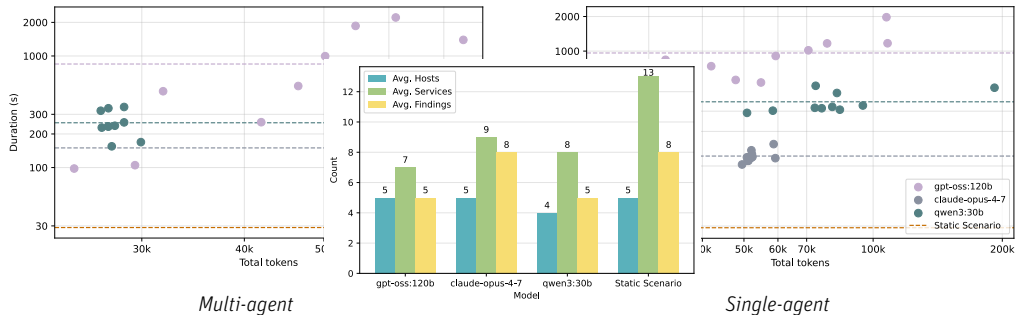
Containerlab – Multi-Agent vs Single-Agent

Tokens vs. duration – multi-agent first, then single-agent



Containerlab – Multi-Agent vs Single-Agent

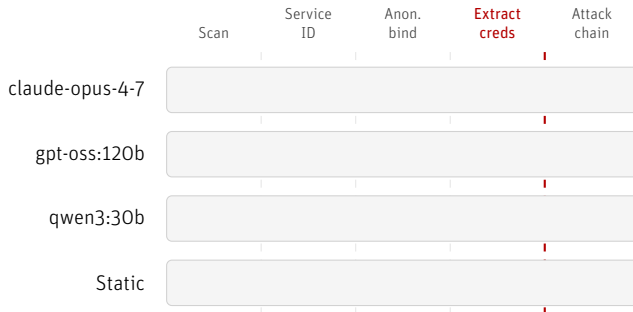
Tokens vs. duration – multi-agent first, then single-agent



- Decomposition: **qwen3:30b** drops tokens **–68%** and duration **–30%**.
- **claude-opus-4-7** keeps identical discovery quality (slight +13% token overhead).

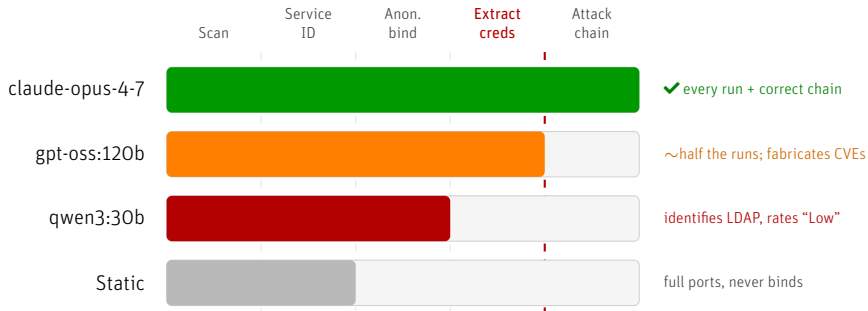
Qualitative Results + Interactive Mode

Q How far does each actor get along the attack chain?



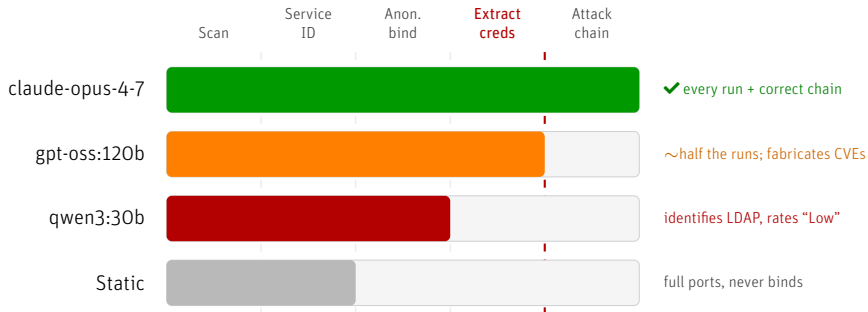
Qualitative Results + Interactive Mode

🔍 How far does each actor get along the attack chain?



Qualitative Results + Interactive Mode

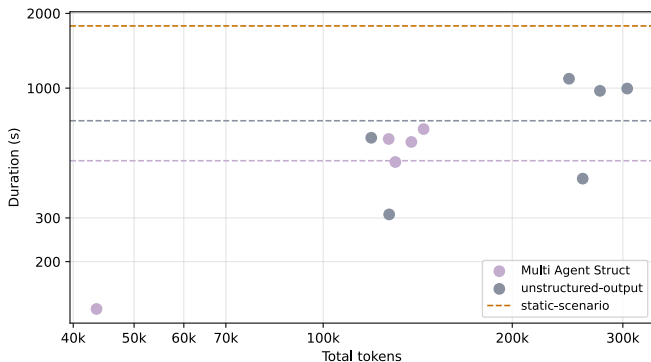
🔍 How far does each actor get along the attack chain?



Trade-off: static = breadth & determinism; AI = interpretive depth (severity-rated assessment, next steps).

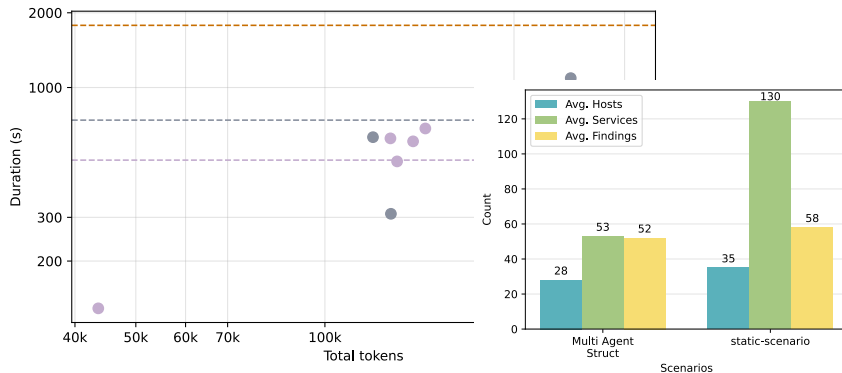
Network Security Lab – Multi-Agent Benchmark

NSlab - Findings



Network Security Lab – Multi-Agent Benchmark

NSlab - Findings



Discussion and Conclusion

Key Findings

Takeaways across both environments

✓ What works

- Even smaller models can autonomously run **standard red-team reconnaissance**
- **Multi-agent decomposition** helps smaller models (−68% tokens for **qwen3:30b**)
- AI delivers **depth**: assessment, prioritized findings, remediation

Key Findings

Takeaways across both environments

✓ What works

- Even smaller models can autonomously run **standard red-team reconnaissance**
- **Multi-agent decomposition** helps smaller models (−68% tokens for **qwen3:30b**)
- AI delivers **depth**: assessment, prioritized findings, remediation

⚠ What limits it

- **High failure rate** – best-case subset
- **Hallucinations**: fabricated CVEs, honeypots

Answering the Research Questions

RQ1 How does an agentic AI scenario perform **compared to a static reference** scenario in network reconnaissance?

Answering the Research Questions

- RQ1** How does an agentic AI scenario perform **compared to a static reference** scenario in network reconnaissance?
- RQ2** To what extent can agentic AI **support operators** during interactive reconnaissance?

Answering the Research Questions

- RQ1** How does an agentic AI scenario perform **compared to a static reference** scenario in network reconnaissance?
- RQ2** To what extent can agentic AI **support operators** during interactive reconnaissance?
- RQ3** To what extent is the implementation **suitable for real-world** deployment?

Answering the Research Questions

- RQ1 How does an agentic AI scenario perform **compared to a static reference** scenario in network reconnaissance?
- RQ2 To what extent can agentic AI **support operators** during interactive reconnaissance?
- RQ3 To what extent is the implementation **suitable for real-world** deployment?
- RQ1 **AI vs. static:** Same discovery power as baseline, no gains in speed/determinism. Adds value via **reasoning and interpretation** (credentials, banner reasoning, severity scoring).

Answering the Research Questions

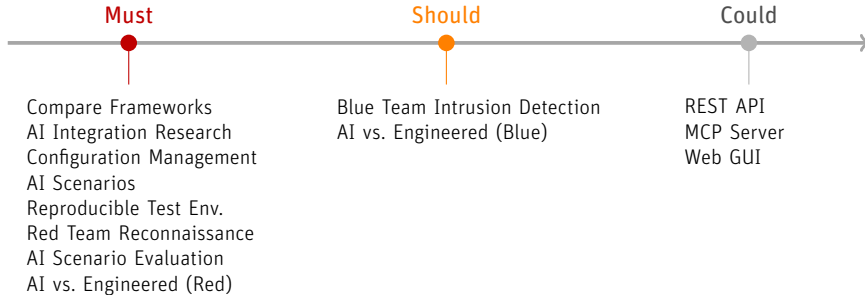
- RQ1 How does an agentic AI scenario perform **compared to a static reference** scenario in network reconnaissance?
- RQ2 To what extent can agentic AI **support operators** during interactive reconnaissance?
- RQ3 To what extent is the implementation **suitable for real-world** deployment?
- RQ1 **AI vs. static:** Same discovery power as baseline, no gains in speed/determinism. Adds value via **reasoning and interpretation** (credentials, banner reasoning, severity scoring).
- RQ2 **Operator support:** Converts scans into structured, severity-ranked output + next steps. **--interactive**
Operator stays in the loop and control.

Answering the Research Questions

- RQ1 How does an agentic AI scenario perform **compared to a static reference** scenario in network reconnaissance?
- RQ2 To what extent can agentic AI **support operators** during interactive reconnaissance?
- RQ3 To what extent is the implementation **suitable for real-world** deployment?
- RQ1 **AI vs. static:** Same discovery power as baseline, no gains in speed/determinism. Adds value via **reasoning and interpretation** (credentials, banner reasoning, severity scoring).
- RQ2 **Operator support:** Converts scans into structured, severity-ranked output + next steps. **--interactive**
Operator stays in the loop and control.
- RQ3 **Deployment:** Scales to ~35 hosts, highlights key risks. **Requires supervision** (reliability limits).

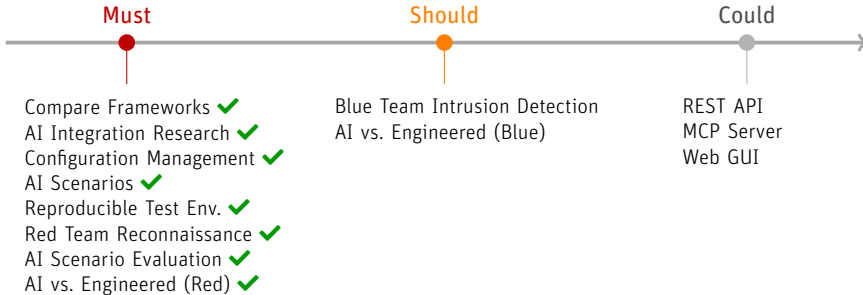
Project Goals

Order of implementation: Must → Should → Could



Project Goals

Order of implementation: Must → Should → Could



Recommendations and Future Work

Recommendations

- **Match model tier to task** – frontier model for autonomous recon
- **Multi-agent** for smaller / self-hosted models
- **Hybrid pipeline:** static breadth + AI depth (RAG Retrieval-Augmented Generation)
- **Human-in-the-loop** + retry on incomplete runs

Future work

- **Dynamic tool calling** – expose only relevant tools
- **Guardrails & sandboxing** – scope restriction for production
- **Robust HITL middleware**
- **Broader attack-chain** coverage beyond reconnaissance

Conclusion

Agentic AI complements deterministic enumeration

- ✓ Using NSAK directly as an **agentic harness** proved effective – the agent autonomously and interactively executes network discovery via LangChain tools.
- ✓ Scripted scenarios excel in correctness and completeness; the agent adds **reasoning and assessment**, limited by model capability.
- ✓ **Multi-agent decomposition** models to digest large context improves models by reducing context complexity and increasing consistency.
- ⚠ Hallucinations remain a limitation – **supervised operation with a human reviewer** is the proposed deployment model.

LLMs make cybersecurity use cases more accessible and cost-effective – with a human in the loop.

Personal Conclusion

Frank

- ContainerLab impressed me – simple to deploy and reconfigure, motivated a home cyber lab
- Enjoyed researching state-of-the-art models, papers and LangChain
- Prefer LLMs mainly for reasoning and recommendations
- Surprised how hard autonomous NSAK deployment in the NSLab was
- Deep respect for academic writing – keeping a consistent “red thread” is hard

Lukas

- Initially skeptical of the usefulness of AI
- We covered a lot of topics we learned at BFH
- A bit of naivety in the planning led to a larger scope
- Keep focusing on the goals and research questions
- AI is here to stay as another engineering tool

Demo: Agentic Reconnaissance in Action